```
class WelcomeController < ApplicationController
  def index
    redis = Redis.new(host: "redis", port: 6379)
    redis.incr "page hits"

    @page_hits = redis.get "page hits"
➤   @hostname = Socket.gethostname
  end
end
```

Now let's output the hostname in our view (app/views/welcome/index.html.erb):

```
<% content_for :head do %>
  <script type="text/javascript">
    document.addEventListener("DOMContentLoaded",function(){
      document.getElementsByTagName('h1')[0].append(' ENHANCED!');
    });
  </script>
<% end %>

<h1>This page has been viewed <%= pluralize(@page_hits, 'time') %>!</h1>
➤ <p>Request handled by host: <b><%= @hostname %></b></p>
```

We've changed our code, so we need to rebuild our image. However, remember that our Docker CLI is currently configured to target our VirtualBox instance. Really, we should switch back to our local Docker installation to build the image, rather than using the swarm instance. Typically, you won't have to remember this, as the images can be built as part of your continuous integration/continuous delivery (CI/CD) pipeline.

Let's switch our Docker CLI back to targeting commands at our local Docker installation. We do this by running:

```
$ eval $(docker-machine env -u)
```

This unsets the environment variables that we'd previously set, which made our Docker CLI target our Docker Machine–created VirtualBox instance.

Now we're good to rebuild our image:

```
$ docker build -f Dockerfile.prod -t robisenberg/myapp_web:prod .
«...»
Successfully built dd61876489b2
Successfully tagged robisenberg/myapp_web:prod
```

As we saw previously, the next step is to push our updated image to a Docker Registry—in our case, Docker Hub:

```
$ docker push robisenberg/myapp_web:prod
```

Now let's switch back to targeting our VirtualBox swarm instance: